

Getting Started: Server Access, Docker, and Kubernetes

This guide walks you through connecting to the course server and running your first container and (later) Kubernetes workloads. Your username and password were sent to you separately — keep them private, do not share them with classmates.

1. Before you start

You will need:

- **Server address:** 164.52.205.84
- **Username:** your smail ID without the @smaill.iitm.ac.in part (e.g. da25m501)
- **Password:** sent to you separately

You do **not** need `sudo` /admin rights for anything in this guide. Your account intentionally does not have them — everything below (Docker, Kubernetes) is designed to work as a normal, unprivileged user.

2. Connecting to the server (SSH)

SSH ("Secure Shell") is how you get a terminal on the remote server from your own laptop.

macOS / Linux

Open the **Terminal** app and run:

```
ssh <username>@164.52.205.84
```

Example:

```
ssh da25m501@164.52.205.84
```

The first time you connect, you'll see a message like:

```
The authenticity of host '164.52.205.84' can't be established.  
Are you sure you want to continue connecting (yes/no)?
```

Type `yes` and press Enter. Then enter your password when prompted (the characters won't appear as you type — that's normal, just type and press Enter).

Windows

Use **Windows Terminal** or **PowerShell** (both ship with `ssh` built in on modern Windows):

```
ssh <username>@164.52.205.84
```

Same prompts as above apply. If you prefer a GUI tool instead, **PuTTY** also works — enter the server address under "Host Name," port `22`, and log in with your username/password when the terminal window opens.

After logging in

You should see a shell prompt like:

```
da25m501@e2e-73-84:~$
```

That confirms you're in. Everything from here on happens inside this remote session, not on your own laptop.

Changing your password (recommended, first login)

```
passwd
```

It will ask for your current password once, then your new password twice. Pick something you'll remember — there's no password recovery flow, so if you forget it you'll need to email the instructor to get it reset.

3. Running Docker (rootless)

Your account already has Docker fully set up in **rootless mode** — Docker runs as *you*, not as an administrator, so nothing you do inside Docker can affect other students' containers or the host system.

Check it's working

```
docker info
```

If you see daemon details (not an error), you're good to go — no extra setup needed.

Run your first container

```
docker run --rm hello-world
```

You should see a "Hello from Docker!" message. `--rm` just cleans up the container after it exits so it doesn't clutter your list.

Common commands

<code>docker ps</code>	<code># containers currently running</code>
<code>docker ps -a</code>	<code># all containers, including stopped ones</code>
<code>docker images</code>	<code># images you've downloaded/built</code>
<code>docker run -it ubuntu bash</code>	<code># start an interactive Ubuntu shell in a container</code>
<code>docker build -t myapp .</code>	<code># build an image from a Dockerfile in the current directory</code>
<code>docker run myapp</code>	<code># run a container from that image</code>
<code>docker stop <container_id></code>	<code># stop a running container</code>
<code>docker rm <container_id></code>	<code># remove a stopped container</code>
<code>docker rmi <image_id></code>	<code># remove an image</code>
<code>docker logs <container_id></code>	<code># view a container's output</code>

Exposing a port from a container

Rootless Docker **cannot bind ports below 1024** (that's a Linux restriction for unprivileged processes, not something specific to your account). Use a port at or above `1024` on the host side:

```
docker run -p 8080:80 nginx
```

This maps container port `80` to host port `8080`. You'd then reach it at `164.52.205.84:8080` from your own machine (assuming the port is open through any firewall/VPN — check with your instructor if it doesn't load).

Building your own images

```
mkdir myproject && cd myproject
cat > Dockerfile <<'EOF'
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
EOF

docker build -t myproject .
docker run --rm myproject
```

Resource limits (read this)

This server is shared across everyone in the course. Your account has a cap of **2 CPUs and 4 GB RAM** for everything you run. If your container gets killed unexpectedly or feels very slow, you may be hitting that limit — check with:

```
docker stats
```

If your assignment genuinely needs more than that, talk to your instructor rather than trying to work around it — the limit exists so that one person's workload can't starve everyone else's.

If Docker seems "down"

Your Docker daemon runs continuously in the background via `systemd`, so this should be rare, but if `docker ps` gives a connection error:

```
systemctl --user status docker      # check status
systemctl --user restart docker     # restart it
```

If it still doesn't come back, email the instructor rather than trying to debug the daemon itself — you don't have the permissions to fix daemon-level issues, and you shouldn't need them.

4. Running experiments on Kubernetes

This section will be finalized once your Kubernetes access (a `kubeconfig` file) is distributed — you'll receive it the same way you received your password. The instructions below are how you'll use it once you have it.

One-time setup

Save the `kubeconfig` file you're given to the standard location:

```
mkdir -p ~/.kube
cp /path/to/your-downloaded-kubeconfig ~/.kube/config
chmod 600 ~/.kube/config
```

Check it works

```
kubectl get pods
```

An empty list (not an error) means it's working — you just don't have any pods running yet. You are scoped to your **own namespace only**; you won't see or be able to affect other students' workloads, and you have a quota of:

- 2 CPU / 4 GB RAM requested, up to 4 CPU / 8 GB RAM burst
- Up to 10 pods at a time

Common commands

```
kubectl get pods                # list your pods
kubectl get all                 # list everything in your
namespace
kubectl apply -f deployment.yaml # create/update from a manifest
file
kubectl logs <pod-name>         # view a pod's logs
kubectl exec -it <pod-name> -- bash # get a shell inside a running
pod
kubectl delete pod <pod-name>    # delete a pod
kubectl describe pod <pod-name>  # detailed status/events (use
this to debug)
```

Example: running something simple

```
# my-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-experiment
spec:
  containers:
  - name: app
    image: python:3.12-slim
    command: ["python", "-c", "print('hello from kubernetes')"]
```

```
kubectl apply -f my-pod.yaml
kubectl logs my-experiment
kubectl delete -f my-pod.yaml
```

If something doesn't work

```
kubectl describe pod <pod-name>
```

This shows *why* a pod is stuck (image pull errors, quota exceeded, etc.) — it's the first thing to check before asking for help.

5. Good practices on a shared server

- This is a shared machine — dozens of classmates are using the same box. Don't run anything intentionally resource-intensive as a "test" (crypto miners, stress tests, etc.).
 - Clean up containers/images/pods you no longer need (`docker system prune` , `kubectl delete`) so you don't run into your own quota unnecessarily.
 - Don't try to access another student's files, containers, or Kubernetes namespace — you don't have permission to, and attempts will be logged.
 - Log out (`exit` or `Ctrl+D`) when you're done with a session; your containers and pods keep running in the background regardless, so you don't need to stay connected.
-

6. Getting help

If something in this guide doesn't work as described, email the instructor with:

1. The exact command you ran
2. The exact error message (copy-paste the text, not a description of it)
3. Your username

That's usually enough to diagnose the issue quickly.